

Projeto Loja de Ferramentas - Avaliação Formativa Noite

Projeto:

Descrição do Projeto:

Elaboração do Escopo e Objetivos do Projeto:

Escopo:

Desenvolver uma loja virtual de ferramentas utilizando o framework Laravel. A loja permitirá a visualização, pesquisa, compra e gestão de ferramentas com funcionalidades tanto para clientes quanto para administradores. As funcionalidades incluirão cadastro e login de usuários, navegação e pesquisa de produtos, gerenciamento de carrinho de compras e pedidos, e administração de produtos e usuários.

Objetivos:

Específicos:

- Implementar um sistema de registro e login que permita aos clientes criar e gerenciar suas contas.
- Desenvolver uma interface intuitiva para navegação por categorias e pesquisa de produtos, facilitando a localização de ferramentas específicas.
- Criar um sistema de carrinho de compras que permita aos clientes adicionar e remover itens, revisar o pedido e concluir a compra com opções de pagamento seguras.
- Desenvolver um painel de administração para gerenciar produtos, categorias e pedidos, além de gerar relatórios de vendas.
- Implementar medidas robustas de segurança para proteger as informações pessoais dos clientes e garantir a integridade das transações financeiras.

Mensuráveis:

O sistema deve permitir o registro e login de no mínimo 500 usuários no primeiro mês após o lançamento.

A taxa de satisfação com a navegação e pesquisa deve ser de pelo menos 90% em testes de usabilidade.

O sistema deve processar e concluir no mínimo 1000 pedidos durante o primeiro mês após o lançamento.

Administradores devem ser capazes de adicionar e gerenciar produtos e pedidos com uma taxa de sucesso de 95% em suas tarefas.

Atingíveis:

- Implementar um sistema de gerenciamento de produtos e pedidos dentro dos recursos e prazos estabelecidos.
- Utilizar tecnologias e ferramentas testadas e comprovadas para garantir eficiência e segurança.
- Treinar a equipe administrativa para operar o painel de forma eficaz e autônoma.

Relevantes:

- Atender à demanda por uma plataforma de e-commerce eficiente para a venda de ferramentas.
- Oferecer serviço de autonomia aos administradores do sistema para cadastro de produtos
- Garantir uma satisfação do uso da plataforma por parte dos clientes na busca por ferramentas e equipamentos
- Proporcionar uma solução prática e escalável que atenda às necessidades dos clientes e administradores, com possibilidade de expansão futura.

Temporais:

- Concluir projeto em 8 semanas.
- Configuração do ambiente e desenvolvimento do sistema de cadastro e autenticação em 2 semanas
- Implementação da navegação, pesquisa de produtos e páginas de detalhes. em 1 semana
- Implementação do painel de administração e funcionalidades administrativas até a 5 semana
- Implantação e Monitoramento até a 7 semana

Planejamento do Projeto:

Cronograma:

1ª Etapa

Planejamento e Análise:

- Definição dos requisitos do projeto e análise das necessidades dos stakeholders.
- Criação de documentação técnica e diagramas de fluxo.

2ª Etapa

Desenvolvimento:

- Configuração do ambiente de desenvolvimento e criação do sistema de cadastro e login.
- Desenvolvimento da interface de navegação, pesquisa e páginas de detalhes dos produtos.
- Implementação das funcionalidades do carrinho de compras e checkout.
- Criação do painel de administração e ferramentas para gestão de produtos e pedidos.
- Implementação de segurança e ajustes finais conforme feedback.

3ª Etapa

Testes e Validação:

- Condução de testes funcionais, de usabilidade e segurança.
- Correção de bugs e refinamento baseado nos resultados dos testes.

4ª Etapa

Implantação e Monitoramento:

- Lançamento da loja virtual em ambiente de produção.
- Monitoramento contínuo do desempenho e suporte técnico pós-lançamento.

Recursos:

Equipe de Desenvolvimento:

- Desenvolvedores Backend e Frontend.
- Designer UI/UX para a criação de uma interface amigável.
- Especialista em Segurança para garantir proteção de dados.
- Testadores/QA para verificar a funcionalidade e usabilidade.

Infraestrutura:

- Servidores de hospedagem (AWS).
- Ferramentas de controle de versão (GitHub).

- Ambiente de desenvolvimento configurado com Laravel e Postgres.

Ferramentas:

- Jira/Trello para gerenciamento de tarefas
- Teams para comunicação interna
- Figma para design de interfaces

Análise de Riscos:

Atrasos no Cronograma:

- **Mitigação:** Implementar um plano de gerenciamento de projeto detalhado e realizar revisões semanais para monitorar o progresso e identificar possíveis desvios.

Problemas de Segurança:

- **Mitigação:** Adotar melhores práticas de segurança e realizar testes de vulnerabilidade regulares para proteger dados sensíveis e transações.

Integração com Sistemas de Pagamento:

- **Mitigação:** Selecionar provedores de pagamento confiáveis e testar a integração com antecedência para garantir a funcionalidade.

Mudanças nos Requisitos:

- **Mitigação:** Estabelecer um processo formal para gerenciamento de mudanças e manter uma comunicação aberta com stakeholders para gerenciar expectativas.

Desempenho Insuficiente do Sistema:

- **Mitigação:** Realizar testes de carga e otimizar o desempenho durante as fases de desenvolvimento e teste para garantir a eficiência do sistema.

Feedback Negativo dos Usuários:

- **Mitigação:** Conduzir testes de usabilidade durante o desenvolvimento e ajustar funcionalidades com base no feedback para melhorar a experiência do usuário.

Desenvolvimento:

Diagrama de Desenvolvimento:

Classes Principais:

- Usuário -
Atributos
 - String - Nome
 - String - Email
 - String - Senha
 - Enum - Eh_AdmMétodos
 - Create -
 - Auth -
 - Update -
- Produto -
Atributos
 - Nome
 - Descrição
 - Preço
 - Quantidade
 - CategoriaMétodos
 - CRUD
- Carrinho -
Atributos

Id_Usuario
Id_Produto
Quantidade Produto
Valor_Total

Métodos:

Adicionar
Remover
Finalizar

- Pagamento

Atributos

Id_Usuário
Id_Carrinho
Valor
Data
Status

Relacionamentos:

Usuario => Produtos

Usuário => Carrinho

Carrinho => Pagamento

Diagrama de Casos de Uso

Usuários

Cadastro
Login
Comprar
CRUD Produtos(ADM)

Produtos

Adicionar e Remover Carrinho

Carrinho

Status Pagamento

Diagrama de Fluxo

Página Inicial(Home)

|

Login -> Fazer Cadastro -> Fazer Login

|

Verificar Tipo de Usuários

|

Cliente -> Ver Produtos -> Carrinho -> Finalizar -> Pagamento

Adm -> CRUD PRodutos -> Dashboard de Vendas

|

Logout -> Home

Passo a Passo do Desenvolvimento

Criar Projeto Laravel

```
composer create-project --prefer-dist laravel/laravel LojaFerramentas
```

Configuração do Banco de Dados

No arquivo .env, configure a conexão com o banco de dados:

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=loja_ferramentas
DB_USERNAME=postgres
DB_PASSWORD=postgres
```

Execute a migração

```
php artisan migrate
```

Modelos e Migrações

Modificar o User Model

```
protected $fillable = [
```

```

        'name',
        'email',
        'password',
        'endereco',
        'telefone',
        'tipo_usuario'
    ];

```

// Adicionar no Final do Model

```

// Verifica se o usuário é um administrador
public function isAdmin()
{
    return $this->tipo_usuario === 'administrador';
}

// Verifica se o usuário é um cliente
public function isClient()
{
    return $this->tipo_usuario === 'cliente';
}

```

Modificar o Migration - User

Adicionar alguns atributos para armazenar informações adicionais. em
database/migrations/xxxx_xx_xx_create_users_table.php

```

Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->timestamp('email_verified_at')->nullable();
    $table->string('password');
    $table->string('endereco')->nullable();
    $table->string('telefone')->nullable();
    $table->enum('tipo_usuario', ['cliente',
'administrador'])->default('cliente');
    $table->rememberToken();
    $table->timestamps();
});

```


Criar e Modificar os Models e Migrations Para Produtos e Carrinho

```
php artisan make:model Produto -m  
php artisan make:model Carrinho -m  
php artisan make:model Pagamento -m
```

Criar o Controller Para User

```
php artisan make:controller UserController
```

Editar o Arquivo App/Http/Controller/UserController.php

Criar os Mecanismos de Autenticação

```
<?php  
  
namespace App\Http\Controllers;  
  
use Illuminate\Http\Request;  
use App\Models\User;  
use Illuminate\Support\Facades\Auth;  
use Illuminate\Support\Facades\Hash;  
  
class UserController extends Controller  
{  
    // Exibir o formulário de login  
    public function showLoginForm()  
    {  
        return view('usuarios.login');  
    }  
  
    // Processar o login do usuário  
    public function login(Request $request)  
    {  
        $credentials = $request->validate([  
            'email' => ['required', 'email'],  
            'password' => ['required'],  
        ]);  
    }  
}
```

```

        if (Auth::guard('web')->attempt($credentials)) {
            $request->session()->regenerate();
            return redirect()->intended('/dashboard');
        }

        return back()->withErrors([
            'email' => 'As credenciais não correspondem aos nossos
registros.',
        ])->onlyInput('email');
    }

    // Exibir o formulário de registro
    public function showRegistroForm()
    {
        return view('usuarios.registro');
    }

    // Processar o registro de um novo usuário
    public function registro(Request $request)
    {
        $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|string|email|max:255|unique:users',
            'password' => 'required|string|min:8|confirmed',
        ]);

        $usuario = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
        ]);

        return redirect('/');
    }

    // Realizar o logout do usuário
    public function logout(Request $request)
    {
        Auth::logout();
    }

```

```

        $request->session()->regenerateToken();
        $request->session()->invalidate();

        return redirect('/');
    }
}

```

Criar as View

views/layouts/app.blade.php -> Super View

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Loja Ferramentas</title>
    {{-- link de css --}}
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.
css" rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMHjY6h
W+ALEwIH" crossorigin="anonymous">
    <link rel="stylesheet" href="/assets/css/style.css">
</head>
<body>

    @include('parts.header')

    {{-- partes das outra páginas que será inserida --}}
    @yield('content')

    @include('parts.footer')

</body>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle

```

```
.min.js"
integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslKleN
7N6jIeHz" crossorigin="anonymous"></script>
</html>
```

views/parts/header.blade.php -> header

```
@if (Auth::check())
    <div>
        <h3>Olá, {{Auth::user()->name }}</h3>
        <h4>{{Auth::user()->tipo_usuario}}</h4>
    </div>
    <form action="/logout" method="post">
        @csrf
        <button type="submit">Logout</button>
    </form>
@else
    <div>
        <ul>
            <li><a href="/login">Login</a></li>
            <li><a href="/registro">Registro</a></li>
        </ul>
    </div>

@endif
```

views/parts/footer.blade.php - > footer

views/usuarios/registro.blade.php

```
{{-- formularios de cadastro --}}
@extends('layouts.app')

@section('content')
    {{-- formulario --}}
    <div class="container">
        <h1>Registrar-se</h1>
        <form method="POST" action="{{ route('usuarios.registro') }}">
```

```

@csrf

<div class="form-group">
    <label for="name">Nome</label>
    <input type="text" name="name" class="form-control" required>
</div>

<div class="form-group">
    <label for="email">Email</label>
    <input type="email" name="email" class="form-control
@error('email')is-invalid @enderror" value="{{old('email')}}" required>
    @error('email')
    <div class="invalid-feedback">{{$message}}</div>
    @enderror
</div>

<div class="form-group">
    <label for="password">Senha</label>
    <input type="password" name="password" class="form-control
@error('password')is-invalid @enderror" required>
    @error('password')
    <div class="invalid-feedback">{{$message}}</div>
    @enderror
</div>

<div class="form-group">
    <label for="password_confirmation">Confirme a Senha</label>
    <input type="password" name="password_confirmation"
class="form-control" required>
</div>

<button type="submit" class="btn
btn-primary">Registrar-se</button>
</form>
</div>

@endsection

```

views/usuarios/login.blade.php

```

@extends('layouts.app')

@section('content')
<div class="container">
    <h1>Login</h1>
    <form method="POST" action="{{ route('usuarios.login') }}">
        @csrf

        <div class="form-group">
            <label for="email">Email</label>
            <input type="email" name="email" class="form-control" required
autofocus>
        </div>

        <div class="form-group">
            <label for="password">Senha</label>
            <input type="password" name="password" class="form-control"
required>
        </div>

        <button type="submit" class="btn btn-primary">Login</button>
    </form>
</div>

@endsection

```

Criar o CRUD de Produto

Definir Atributos do Model Produto

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Produto extends Model

```

```
{
    use HasFactory;
    protected $fillable = [
        'nome',
        'descricao',
        'categoria',
        'preco',
        'quantidade',
        'img'
    ];
}
```

Modificar a Migration create_produtos_table

```
Schema::create('produtos', function (Blueprint $table) {
    $table->id();
    $table->string('nome');
    $table->text('descricao');
    $table->string('categoria');
    $table->decimal('preco', 8, 2);
    $table->integer('quantidade');
    $table->string('img')->nullable();
    $table->timestamps();
});
```

Criar o Classe ProdutoController

```
php artisan make:controller ProdutoController --resource
```

Dentro de App\Http\Controller\ProdutoController.php

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Models\Produto;

class ProdutoController extends Controller
{
```

```

/**
 * Display a listing of the resource.
 */
//lista todos os produtos
public function index()
{
    $produtos = Produto::all();
    return view('produtos.index',compact('produtos'));
}

/**
 * Show the form for creating a new resource.
 */
//abre o formulario de cadastro
public function create()
{
    return view('produtos.create');
}

/**
 * Store a newly created resource in storage.
 */
// envia o formulario de cadastro
public function store(Request $request)
{
    $request->validate([
        'nome'=> 'required|string|max:255',
        'descricao'=> 'required',
        'categoria'=> 'required',
        'quantidade'=> 'required|numeric',
        'preco'=> 'required|numeric',
    ]);

    Produto::create($request->all());

    return redirect()->route('produtos.index')->
        with('sucesso','Produto Criado com Sucesso');
}

```



```

/**
 * Show the form for editing the specified resource.
 */
public function edit(Produto $produto)
{
    return view('produtos.edit', compact('produto'));
}

/**
 * Update the specified resource in storage.
 */
public function update(Request $request, Produto $produto)
{
    $request->validate([
        'nome'=> 'required|string|max:255',
        'descricao'=> 'required',
        'categoria'=> 'required',
        'quantidade'=> 'required|numeric',
        'preco'=> 'required|numeric',
    ]);

    $produto->update($request->all());

    return redirect()->route('produtos.index')->
        with('sucess', 'Produto Atualizado com Sucesso');
}

/**
 * Remove the specified resource from storage.
 */
public function destroy(Produto $produto)
{
    $produto->delete();

    return redirect()->route('produtos.index')->
        with('sucess', 'Produto Deletado com Sucesso');
}
}

```

Views Para Produtos

views/produtos/index.blade.php - php artisan make:view produtos.index

```
@extends('layouts.app')

@section('content')

<div class="container">
    <h1 class="my-4">Produtos</h1>

    @if ($message = Session::get('success'))
        <div class="alert alert-success">
            <p>{{ $message }}</p>
        </div>
    @endif

    <a class="btn btn-success mb-2" href="{{ route('produtos.create') }}">
Criar Novo Produto</a>

    <table class="table table-bordered">
        <tr>
            <th>No</th>
            <th>Nome</th>
            <th>Descrição</th>
            <th>Categoria</th>
            <th>Quantidade</th>
            <th>Preço</th>
            <th width="280px">Ação</th>
        </tr>
        @foreach ($produtos as $produto)
            <tr>
                <td>{{ $loop->iteration }}</td>
                <td>{{ $produto->nome }}</td>
                <td>{{ $produto->descricao }}</td>
                <td>{{ $produto->categoria }}</td>
                <td>{{ $produto->quantidade }}</td>
                <td>{{ $produto->preco }}</td>
                <td>
                    <form action="{{ route('produtos.destroy', $produto->id)
}} " method="POST">
```

```

                <a class="btn btn-info" href="{{
route('produtos.show', $produto->id) }}">Mostrar</a>
                <a class="btn btn-primary" href="{{
route('produtos.edit', $produto->id) }}">Editar</a>

                @csrf
                @method('DELETE')
                <button type="submit" class="btn
btn-danger">Deletar</button>
            </form>
        </td>
    </tr>
@endforeach
</table>
</div>
@endsection

```

views/produtos/create.blade.php - php artisan make:view produtos.create

```

@extends('layouts.app')

@section('content')
    <div class="container">
        <h1 class="my-4">Criar Produto</h1>

        @if ($errors->any())
            <div class="alert alert-danger">
                <strong>Oops!</strong> Houve alguns problemas com sua
                entrada.<br><br>
                <ul>
                    @foreach ($errors->all() as $error)
                        <li>{{ $error }}</li>
                    @endforeach
                </ul>
            </div>
        @endif

        <form action="{{ route('produtos.store') }}" method="POST">
            @csrf

```

```

        <div class="form-group">
            <label for="nome">Nome:</label>
            <input type="text" name="nome" class="form-control"
placeholder="Nome">
        </div>

        <div class="form-group">
            <label for="descricao">Descrição:</label>
            <textarea name="descricao" class="form-control"
placeholder="Descrição"></textarea>
        </div>

        <div class="form-group">
            <label for="categoria">Categoria:</label>
            <input type="text" name="categoria" class="form-control"
placeholder="Categoria">
        </div>

        <div class="form-group">
            <label for="quantidade">Quantidade:</label>
            <input type="number" name="quantidade"
class="form-control" placeholder="Quantidade">
        </div>

        <div class="form-group">
            <label for="preco">Preço:</label>
            <input type="text" name="preco" class="form-control"
placeholder="Preço">
        </div>

        <button type="submit" class="btn btn-primary">Enviar</button>
    </form>
</div>
@endsection

```

views/produtos/edit.blade.php - php artisan make:view produtos.edit

```
@extends('layouts.app')
```

```

@section('content')
    <div class="container">
        <h1 class="my-4">Editar Produto</h1>

        @if ($errors->any())
            <div class="alert alert-danger">
                <strong>Oops!</strong> Houve alguns problemas com sua
                entrada.<br><br>
                <ul>
                    @foreach ($errors->all() as $error)
                        <li>{{ $error }}</li>
                    @endforeach
                </ul>
            </div>
        @endif

        <form action="{{ route('produtos.update', $produto->id) }}"
method="POST">
            @csrf
            @method('PUT')

            <div class="form-group">
                <label for="nome">Nome:</label>
                <input type="text" name="nome" class="form-control"
value="{{ $produto->nome }}">
            </div>

            <div class="form-group">
                <label for="descricao">Descrição:</label>
                <textarea name="descricao" class="form-control">{{
$produto->descricao }}</textarea>
            </div>

            <div class="form-group">
                <label for="categoria">Categoria:</label>
                <input type="text" name="categoria" class="form-control"
value="{{ $produto->categoria }}">
            </div>

            <div class="form-group">

```

```

        <label for="quantidade">Quantidade:</label>
        <input type="number" name="quantidade"
class="form-control" value="{{ $produto->quantidade }}">
    </div>

    <div class="form-group">
        <label for="preco">Preço:</label>
        <input type="text" name="preco" class="form-control"
value="{{ $produto->preco }}">
    </div>

    <button type="submit" class="btn btn-primary">Enviar</button>
</form>
</div>
@endsection

```

Criar a Rota para Produtos

```

use App\Http\Controllers\ProdutoController;
use App\Http\Middleware\ProdutosMiddleware;

//rota para Produtos
Route::resource('produtos', ProdutoController::class)->
middleware(ProdutosMiddleware::class);

```

Modificando o header.blade.php

```

@if (Auth::check())
    <div>
        <h3>Olá, {{Auth::user()->name }}</h3>
        <h4>{{Auth::user()->tipo_usuario}}</h4>
    </div>
    <form action="/logout" method="post">
        @csrf
        <button type="submit">Logout</button>
    </form>
    @if(Auth::user()->isAdmin())
        <div>

```

```

        <a href="/produtos"><h3>Dashborad Produtos - Adm</h3></a>
    </div>
@endif
@else
    <div>
        <ul>
            <li><a href="/login">Login</a></li>
            <li><a href="/registro">Registro</a></li>
        </ul>
    </div>

@endif

```

Criar as Views / Controller / Routes para Home, Dashboard

views/home.blade.php

Criar Home

```

@extends('layouts.app')

@section('content')
    <div id="productCarousel" class="carousel slide"
    data-bs-ride="carousel">
        <div class="carousel-inner">
            @foreach ($produtos as $index => $produto)
                <div class="carousel-item {{ $index == 0 ? 'active' : ''
            }}">
                    nome }}">
                    <div class="carousel-caption d-none d-md-block">
                        <h5>{{ $produto->nome }}</h5>
                        <p>{{ $produto->descricao }}</p>
                        <p>Preço: R$ {{ $produto->preco }}</p>
                    </div>
                </div>
            @endforeach
        </div>
    </div>

```

```

        <button class="carousel-control-prev" type="button"
data-bs-target="#productCarousel" data-bs-slide="prev">
        <span class="carousel-control-prev-icon"
aria-hidden="true"></span>
        <span class="visually-hidden">Previous</span>
    </button>
    <button class="carousel-control-next" type="button"
data-bs-target="#productCarousel" data-bs-slide="next">
        <span class="carousel-control-next-icon"
aria-hidden="true"></span>
        <span class="visually-hidden">Next</span>
    </button>
</div>
@endsection

```

Criando o HomeController.php

```
php artisan make:controller HomeController
```

código para HomeController.php

```

<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Models\Produto;

class HomeController extends Controller
{
    public function index()
    {
        // Pegue os 5 produtos mais recentes, por exemplo
        $produtos = Produto::take(5)->get();
        return view('home', compact('produtos'));
    }
}

```

views/usuarios/dashboard.blade.php


```

@extends('layouts.app')

@section('content')
    <h1>Dashboard de Produtos</h1>

    <form method="GET" action="{{ route('dashboard') }}">
        <input type="text" name="search" placeholder="Pesquisar
produtos..." value="{{ request('search') }}">
        <button type="submit">Pesquisar</button>
    </form>

    <div class="row">
        @foreach ($produtos as $produto)
            <div class="col-md-4">
                <div class="card">
                    nome }}">
                    <div class="card-body">
                        <h5 class="card-title">{{ $produto->nome }}</h5>
                        <p class="card-text">{{ $produto->descricao }}</p>
                        <p class="card-text">{{ $produto->categoria }}</p>
                        <p class="card-text">Preço: R$ {{ $produto->preco
}}</p>

                        <a href="{{ route('produtos.show', $produto->id)
}}" class="btn btn-primary">Ver Produto</a>
                    </div>
                </div>
            </div>
        @endforeach
    </div>
@endsection

```

Criar o Controller Para Dashboard

php artisan make:controller DashboardController

```
<?php
```

```
namespace App\Http\Controllers;
```

```

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Models\Produto;

class DashboardController extends Controller
{
    public function index(Request $request)
    {
        $search = $request->input('search');
        $produtos = Produto::when($search, function ($query, $search) {
            return $query->where('nome', 'like', "%{$search}%")
                ->orWhere('descricao', 'like', "%{$search}%")
                ->orWhere('categoria', 'like', "%{$search}%");
        })->get();
        /*
        $produtos = Produto::where('nome', 'like', "%{$search}%")
            ->orWhere('descricao', 'like', "%{$search}%")
            ->orWhere('categoria', 'like', "%{$search}%")->get();
        */

        return view('usuarios.dashboard', compact('produtos'));
    }
}

```

Route para Dashboard

```

// Rota para página interna
Route::get('/dashboard', [DashboardController::class, 'index'])
->middleware('auth')->name('dashboard');

```

Criar o Show para Produtos

php artisan make:view produtos.show

código

```

@extends('layouts.app')

@section('content')
    <div class="container">
        <div class="row">
            <div class="col-md-6">
                nome }}">
            </div>
            <div class="col-md-6">
                <h2>{{ $produto->nome }}</h2>
                <p>{{ $produto->categoria }}</p>
                <p>{{ $produto->descricao }}</p>
                <p>Preço: R$ {{ $produto->preco }}</p>

                <form method="POST" action="{{ route('carrinho.add',
$produto->id) }}">
                    @csrf
                    <button type="submit" class="btn
btn-primary">Adicionar ao Carrinho</button>
                </form>
            </div>
        </div>
    </div>
@endsection

```

Criar o Carrinho

Criar Carrinho Model

Criar Carrinho Migration

Criar CarrinhoController